ENGINEERING OPTIMIZATION

Kanpur Version of Zoutendijk Method for Constrained Nonlinear optimization

Rajarshi Dasgupta[a], Monojit Layek[b] and Bhaskar Dasgupta[b]

[a]Department of Computational and Data Sciences, IISc Bangalore—560012, India
[b]Department of Mechanical Engineering, IIT Kanpur—208016, India

ARTICLE HISTORY
Compiled 4th June 2024

Abstract
This study introduces the "Kanpur version of the Zoutendijk Method", a novel adaptation of the Zoutendijk Feasible Direction algorithm specifically tailored for constrained nonlinear optimization problems. The approach strategically reduces the dimension of the linear subproblem in situations with fewer active constraints than variables. Additionally, it incorporates extra measures to effectively handle equality constraints and infeasible starting points, thereby enhancing the algorithm's versatility and effectiveness. Extensive test studies on diverse benchmark problems provide compelling evidence, showcasing remarkable improvements in the convergence rate and algorithm resilience. Consequently, this research makes a significant contribution to the field of constrained optimization by presenting an improved variant of the Zoutendijk algorithm, well equipped to solve a wide range of constrained optimization problems efficiently.

KEYWORDS
Constrained Nonlinear optimization; K-Zoutendijk Algorithm; Feasible Direction Method; Nonlinear Programming;

1. Introduction

Nonlinear programming problems are pervasive in numerous scientific disciplines and industries, ranging from engineering and economics to computer science and operations research. In these domains, the optimization process is pivotal in guiding decision-making and resource allocation, making it essential to find efficient solutions to these optimization problems. The objective is to optimize a given objective function while satisfying a set of constraints, a challenging task that demands sophisticated methodologies to attain optimal results.

Over the years, extensive research has been devoted to developing optimization techniques that can efficiently solve nonlinear programming problems. One such method that has gained significant attention is the Feasible Direction method (Zoutendijk, 1960), which seeks to find the descent direction for minimization problems. This method involves iteratively moving from one point to another while ensuring that each step remains within the feasible space, following such a descent direction.

CONTACT AUTHOR: Email: dasgupta@iitk.ac.in

The Feasible Direction method (Zoutendijk, 1966) for minimization is founded on the observation that, if the point lies on the constraint surface, the inner product function gradient with the descent-feasible direction and the inner product of an active constraint function with the descent-feasible direction both are negative. Through the introduction of a margin and creating a Linear Programming sub-problem, it becomes possible to determine a descending feasible direction by optimizing the margin, as explained in Appendix A.

This research aims to propose an alternative strategy that strategically reduces the dimension of the linear subproblem in most cases where the number of active constraints is less than the number of variables. Additionally, the introduction of a strategy enhances the algorithm's versatility, enabling effective handling of equality constraints and infeasible starting points. The overall objective is to significantly improve the performance of the Zoutendijk Feasible Direction algorithm by consuming less CPU time where as converging in better optima, making it more efficient and robust in solving constrained nonlinear optimization problems.

2. Methodology

We aim to determine a local minimum, of the function $f(\mathbf{x})$, where $\mathbf{x}$ is a vector with elements $x_i \in \mathbb{R}$. The optimization problem is subject to a set of inequality constraints $\mathbf{g}(\mathbf{x}) \leq 0$ that must be satisfied for a valid solution. The goal of the optimization is to minimize $f(\mathbf{x})$ while satisfying all constraints and achieving an optimal solution. The optimization problem can be formulated as follows:

$$\begin{aligned} &\text{minimize} && f(\mathbf{x}), && \forall x_i \in \mathbb{R}; \\ &\text{subject to} && g_p(\mathbf{x}) \leq 0, && p = 1, 2, 3 \ldots; \end{aligned} \tag{1}$$

(1) In constrained minimization, the feasibility of a given direction plays a crucial role. Zoutendijk, 1960 used the concept that a feasible direction must satisfy both the descent condition ($\nabla f^T \mathbf{d} \leq 0$) and the feasibility condition ($\nabla \mathbf{g}^T_{\text{active}} \mathbf{d} \leq 0$). A usable direction satisfies these criteria.

(2) When the current point is interior, say in the k-th iteration the usable direction will be the steepest descent direction ($-\nabla f(\mathbf{x}_k)$) to reach a lower objective function value.

(3) In any of the inequality constraints are active at that point $\mathbf{x}_k$. The negative gradients of the ∇f and $\nabla \mathbf{g}_{active}$ define a feasible cone, depicted by the angular interval θ in Figure 1 for a single active constraint in a 2-d solution space.

(4) In this context, the primary objective is to optimize both the feasible direction within the cone and the step length simultaneously. By doing so, we ensure that the next iterate remains within the feasible space, thus maintaining the feasibility of the solution throughout the optimization process.

(5) Introducing a margin $\beta \geq 0$ for $-\nabla f^T \mathbf{d}$ as well as $-\nabla \mathbf{g}^T \mathbf{d}$ allows us to identify the optimal usable direction within the feasible cone by maximising β.

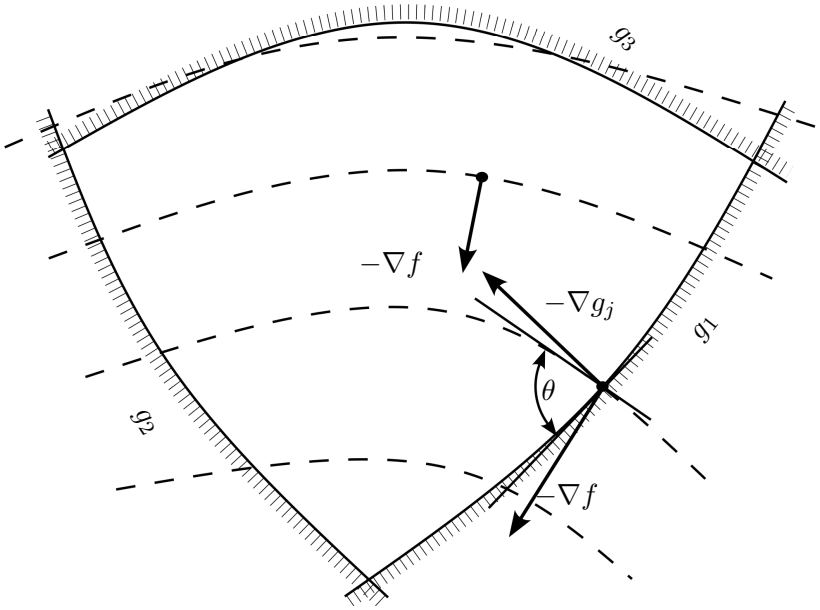


Figure 1. Illustration of Feasible Directions in Constrained Optimization: No Active vs. Active Constraints

2.1. *Mathematical Modification*

At a given point $\mathbf{x}_k$, the active set J represents the set of active constraints. Here, the cardinality of J is m, and $m \leq n$, we look for a feasible direction to identify the optimal solution.

(1) A good feasible direction $\mathbf{d}$ (rather than applying constraints as in the original Zountendijk method, as summarised in Appendix A) is obtained by taking a linear combination of negative gradients of the objective function and active constraints,

$$\mathbf{d} = -c_0 \nabla f - \sum_{j=0}^{m} c_j \nabla g_j = -\mathbf{W}\mathbf{C}, \quad j \in J;$$

where
$$\mathbf{W} = \begin{bmatrix} \nabla f & \nabla \mathbf{g}_{active} \end{bmatrix}, \quad \in \mathbb{R}^{\mathrm{n}\times(\mathrm{m}+1)};$$
$$\mathbf{C} = \begin{bmatrix} c_0 & c_1 & c_2 & \cdots & c_m \end{bmatrix}^T, \quad \mathbf{C} \geq 0 \quad \& \quad \in \mathbb{R}^{(\mathrm{m}+1)}.$$

(2) To accomplish the optimal usable direction, we must maximize the margin (β) from the boundary while simultaneously satisfying the descent $\nabla f^T \mathbf{d} \leq -\beta$ and feasibility $\nabla \mathbf{g}_{active}^T \mathbf{d} \leq -\beta$ conditions,

$$\begin{aligned} &\text{maximize} \quad \beta \\ &\text{subject to} \quad \begin{bmatrix} -\mathbf{W}^T\mathbf{W} & \mathbf{e} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{C} \\ \beta \end{bmatrix} \leq 0, \\ &\qquad\qquad\qquad\qquad\qquad\quad \beta \geq 0, \\ &\qquad\qquad\qquad\qquad\qquad 0 \leq \mathbf{C} \leq \mathbf{e} \end{aligned} \tag{2}$$

where
$$\mathbf{e} = \begin{bmatrix} 1 & 1 & 1 & \ldots \end{bmatrix}^T \quad \in \mathbb{R}^{(\mathrm{m}+1)}.$$

(3) Equation 2 represents a linear programming problem with $m+2$ variables and $m+1$ constraints and $2m+3$ bounds. Using methods such as the simplex method or the interior point method, the optimal values for the variables $\mathbf{C}$ and β can be determined, providing the usable direction,

$$\mathbf{d} = -\mathbf{WC} \quad \in \mathbb{R}^{\mathrm{n}}\,.$$

(4) Afterwards, a line search is conducted along $\mathbf{d}$ using methods such as the golden section or interval halving method to determine an appropriate value for α and α^u represents the upper bound of the line search through the potential of activation of currently 'inactive' constraints.

$$\alpha = \min_{\alpha} f(\mathbf{x} + \alpha\mathbf{d}) \quad \text{where, } \alpha \in (0, \alpha^u)$$

(5) This value is then utilised to compute the next point using the formula,

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha\mathbf{d}$$

The algorithm terminates either when $\|f(\mathbf{x}_k) - f(\mathbf{x}_{k-1})\|$ converges to a specified threshold (η) and satisfies the $\mathbf{E}_{KKT}$ condition (appendix B), or when the maximum number of iterations is reached. The Kanpur Zoutendijk algorithm iteratively refines the solution to maximize the margin while ensuring feasibility.

2.2. *Handling Equality Constraints*

The inability to effectively manage equality constraints is a weakness of the Zoutendijk approach. Problems arise for line search when an $\mathbb{R}^n$ dimensional domain is subject to one or more equality constraints, reducing the dimensionality to $\mathbb{R}^{n-1}$ or less. As shown in the following figure 2, one possible approach to this problem is to transform the equality constraints $h(\mathbf{x})$ into two inequality constraints $g(\mathbf{x}) \pm \eta$, thereby establishing a trust region as in Mehta and Dasgupta, 2012.

To get better outcomes, we use a recursive strategy in which we gradually shrink the trust region by reducing η by a factor γ as the algorithm converges (as shown in the figure 2).

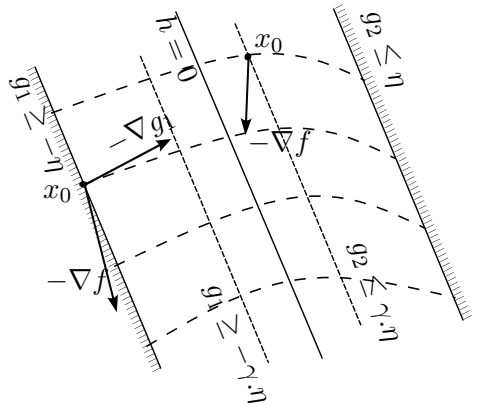


Figure 2. Enhancing Line Search Performance through Conversion of Equality Constraints to Trust Regions

2.3. *Infeasible Starting Point*

It is essential to move towards the feasible region if it begins at an infeasible point, as it will not function properly in an infeasible domain. We use a method that involves finding the nearest violated constraint and performing a descent search against the gradient $(-\nabla g_i(\mathbf{x}_0))$ of the violated constraint, to achieve this goal as shown in the figure 3. This procedure is iterated upon until the violated constraint's viable region is reached. By using a recursive strategy, we can guarantee that all constraints are met, and eventually, reach feasibility as we optimize.

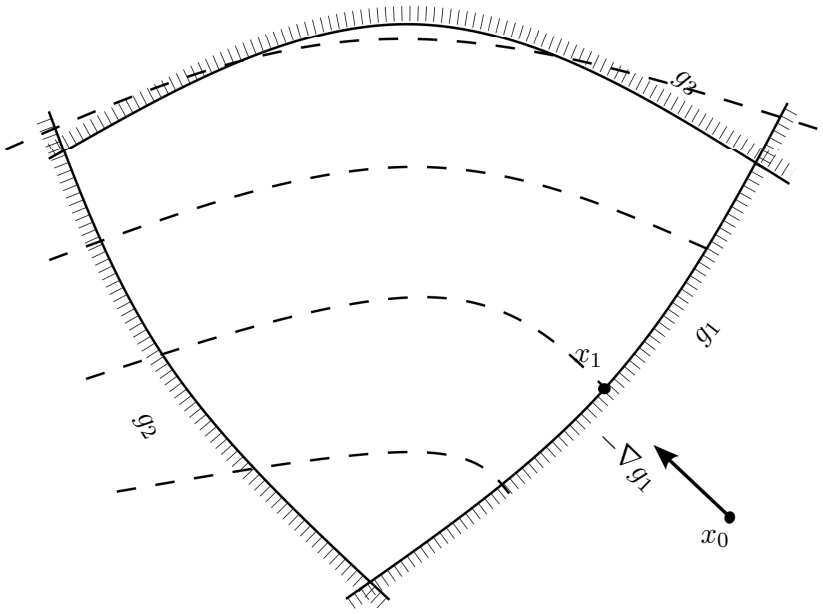


Figure 3. Handling Infeasible Starting Points and Ensuring Feasibility through Constraint Descent

2.4. *Algorithm*

Algorithm 1: Initial point infeasibility checking

Input: $\mathbf{x}, \mathbf{g}, \eta$
Output: $\mathbf{x}_{feasible}$

if $(\forall \mathbf{g}(\mathbf{x_{init}}) \leq 0)$ **then**
  return $\mathbf{x}_{feasible} = \mathbf{x}$ `// Feasibility condition satisfies`
else
  Set, $i = \text{argmin}(\forall \mathbf{g}(\mathbf{x}) > 0)$ `// Closest violated constraint index`
  Set, $l = 0$ and $\mathbf{d} = -\nabla g_i(\mathbf{x})$
  Set, $\phi(l) = g_i(\mathbf{x} + l\mathbf{d})$
  while $\phi(l) \leq \eta$ **do**
    $l = l - \frac{\phi(l)}{\phi'(l)}$ `// Solution along descent direction`
  Set next point to $\mathbf{x} = \mathbf{x} + l\mathbf{d}$
  Repeat for next **i** `// Recursively satisfying all constraints`

Algorithm 2: Kanpur Zoutendijk

Input: f_{obj}, $\mathbf{g}$, $\mathbf{x}_{init}$, $\eta = tolerance$, $N = max\ iteration$
Output: $\mathbf{x}_{optimal}$

Set, iteration count $i = 0$ and $flag = False$ `// Initialize variables`
Set, $\mathbf{x}_i$ = Algorithm 1 $(\mathbf{x}_{init}, \mathbf{g}, \eta)$, `// Initial point may be Infeasible`

`/* Start a loop, runs until the condition is satisfied */`
while $i \leq N$ **do**
  Find the active sets $\mathbf{g}_{active}$ at $\mathbf{x}_i$
  Calculate the gradient of the objective function $f_{grad} = \nabla f_{obj}(\mathbf{x}_i)$
  Calculate the gradient of the active sets $\mathbf{g}_{grad} = \nabla \mathbf{g}_{active}(\mathbf{x}_i)$
  if $\mathbf{g}_{grad}$ *is empty* **then**
    Set $\mathbf{W} = f_{grad}$
  else
    Set $\mathbf{W} = \begin{bmatrix} f_{grad} & \mathbf{g}_{grad} \end{bmatrix}$
  `/* Solve LPP using Simplex Method to compute coefficients */`

$$\begin{aligned} & \text{maximize} && \beta \\ & \text{subject to} && \begin{bmatrix} -\mathbf{W}^T\mathbf{W} & \mathbf{e} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{C} \\ \beta \end{bmatrix} \leq 0, \\ & && 0 \leq \mathbf{C} \leq 1, \\ & && \beta \geq 0 \end{aligned} \tag{3}$$

  Compute $\mathbf{d} = -\mathbf{WC}$
  `/* Line search using Golden Section Method along` $\mathbf{d}$ `*/`
  Compute $\alpha = \min_\alpha f(\mathbf{x} + \alpha\mathbf{d})$ where, $\alpha \in (0, \alpha^u)$
  Update next iterates using, $\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha\mathbf{d}$ $\ni \mathbf{x}_{i+1}$ is interior point.
  if $\|f(\mathbf{x}_{i+1}) - f(\mathbf{x}_i)\| \leq \eta \wedge \|\mathbf{E}_{KKT}\| \approx 0$ **then**
    Set $\mathbf{x}_{optimal} = \mathbf{x}_{i+1}$ and $flag \rightarrow True$
    return $flag$
  else
    $i \rightarrow i + 1$

  return $flag$ `// Process aborted due to maximum iteration reached`

3. Comparison with the original Zoutendijk Method

After conducting a thorough comparison with the Zoutendijk method, we proceeded to assess the performance of the Kanpur Zoutendijk algorithm on various optimization problems. Through a comprehensive analysis, we examined the algorithm's behaviour and effectiveness, specifically focusing on its application to standard benchmark functions. The detailed scrutiny of its performance on these functions provided valuable insights into its efficacy in tackling nonlinear optimization tasks, as demonstrated.

3.1. *Two-Dimensional Problems*

3.1.1. Ackley's Path Function

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = -20e^{-0.2\sqrt{\frac{(x_1^2+x_2^2)}{2}}} - e^{\frac{\cos(2\pi x_1)+\cos(2\pi x_2)}{2}} + 20 + e \\ \text{subject to} \quad & g_1(\mathbf{x}) = 5x_1^3 + 10x_2^2 - 0.5 \leq 0, \\ & g_2(\mathbf{x}) = 5x_1^3 - 4x_2^2 - 0.001 \geq 0, \\ & -1 \leq x_i \leq \infty, \quad i = 1, 2 \end{aligned} \tag{4}$$

The performance of both algorithms on problem 4 (Dasgupta et al., 2013) is evaluated. Figure 4 depicts the infeasible starting point for both algorithms in the $[1, 0.5]$ region. The Zoutendijk algorithm took fewer iterations, although the Kzoutendijk algorithm exhibited faster runtime. Performance details are provided in Table 1.

Table 1. Performance Comparison & optimal outputs on Ackley's Path Function

Method	$\mathbf{x}_{optimal}$	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	[0.0588 -0.0018]	0.255771	1.7114e-05	8	0.170143
KZoutendijk	[0.0587 -0.0009]	0.255519	9.285e-07	9	0.053796

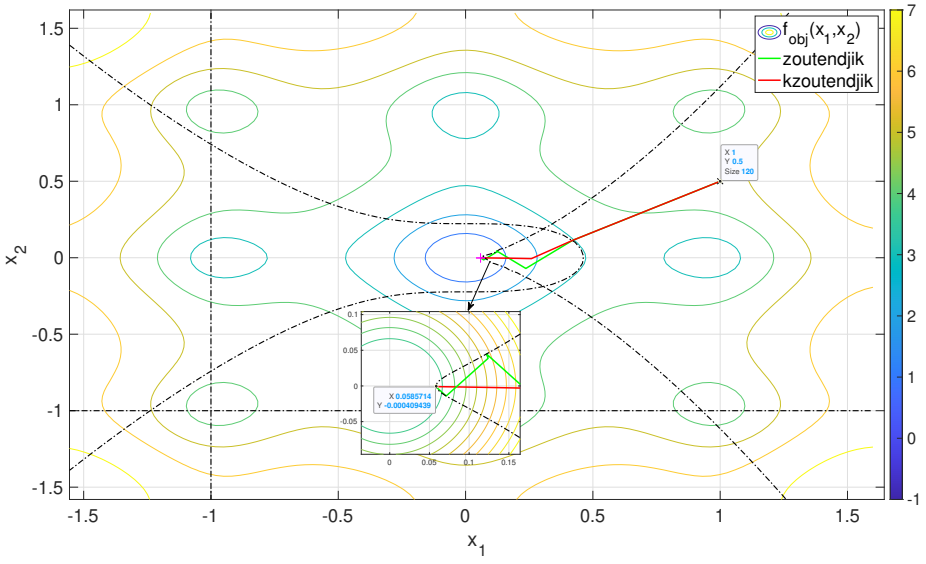


Figure 4. optimization of the Constrained Ackley's path function: Zoutendijk vs. Kzoutendijk Algorithms

3.1.2. Himmelblau Function

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2 \\ \text{subject to} \quad & g_1(\mathbf{x}) = (x_1 - 2)^4 - 4x_2 + 3 \leq 0, \\ & g_2(\mathbf{x}) = x_1 - (x_2 - 2)^4 - 0.5 \geq 0, \\ & 0 \leq x_1 \leq \infty, \\ & 0 \leq x_2 \leq \infty \end{aligned} \tag{5}$$

In problem 5 (Dasgupta et al., 2013), we examine the constrained Himmelblau function, with the starting point for both algorithms in an infeasible region at $[0.5, 0.5]$. Interestingly, this time the Kzoutendijk algorithm required fewer iterations to converge and demonstrated almost twice the speed compared to the previous algorithm.

A comprehensive analysis of both algorithms' performance is facilitated by the data summarized in Table 2 and visualized in Figure 5.

Table 2. Performance Comparison & optimal outputs on Himmelblau Function

Method	$\mathbf{x}_{optimal}$	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	[3.0000 2.0000]	0.000000	3.9404e-37	16	0.082253
KZoutendijk	[3.0000 2.0000]	0.000000	3.9404e-37	11	0.049734

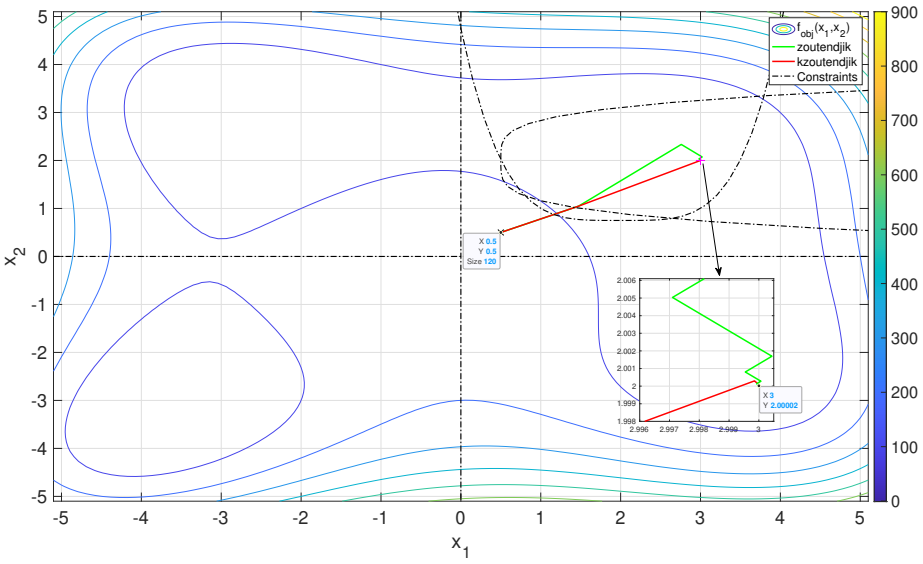


Figure 5. optimization of the Constrained Himmelblau Function: Zoutendijk vs. Kzoutendijk Algorithms

3.1.3. Sixhump Camel Function

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = 4x_1^2 - 2.1x_1^4 + \frac{x_1^6}{3} + x_1x_2 - 4x_2^2 + 4x_2^4 \\
\text{subject to} \quad & g_1(\mathbf{x}) = 35e^{-0.5x_1} + 10x_2 - 50 \leq 0, \\
& g_2(\mathbf{x}) = e^{-x_2} + 3x_1 \geq 0, \\
& -1 \leq x_1 \leq \infty, \\
& 0 \leq x_2 \leq \infty
\end{aligned} \tag{6}
$$

The Problem 6 (Dasgupta et al., 2013) is characterized by two variables and subject to two nonlinear constraints. The analysis of the algorithm's performance started from the initial point $[0, 0]$.

Remarkably, the current algorithm achieved convergence within 7 iterations, outperforming the original method, which required 11 iterations and had a run time twice as much. The results of the found solutions are presented in table 3, and figure 6 visually showcases the method's generational development for this particular function.

Table 3. Performance Comparison & optimal outputs on Sixhump Camel Function

Method	$\mathbf{x}_{optimal}$	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	[-0.1628 0.7221]	-1.011203	3.6679e-11	11	0.065920
KZoutendijk	[-0.1627 0.7224]	-1.011200	2.7532e-11	7	0.036092

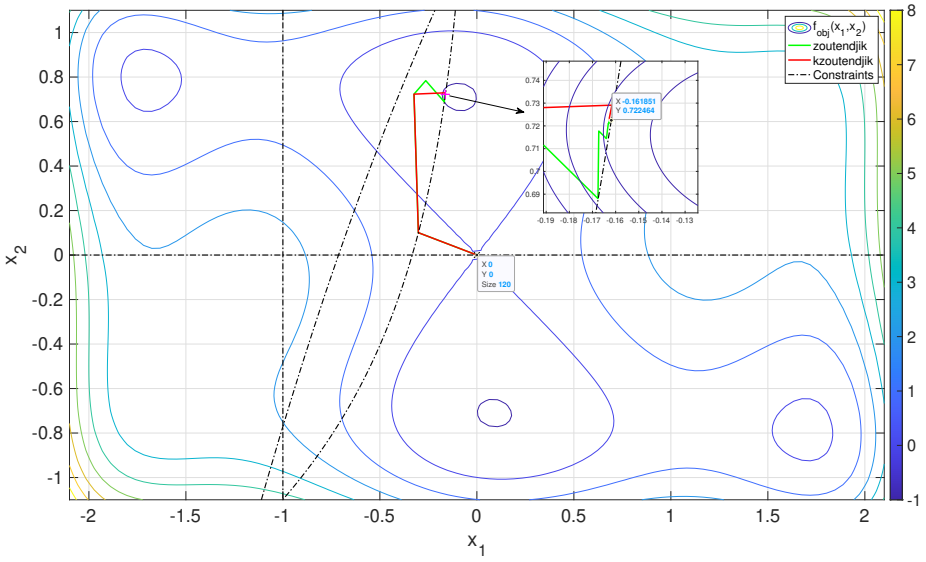


Figure 6. optimization of the Constrained Six-Hump Camel Function: Zoutendijk vs. Kzoutendijk Algorithms

3.1.4. Guilin hills Function

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = 3 + \frac{3(x_1+9)}{(x_1+10)}\sin\left(\frac{\pi}{1.08-x_1}\right) + \frac{1.5(x_2+9)}{(x_2+10)}\sin\left(\frac{\pi}{1.05-x_2}\right) \\
\text{subject to} \quad & g_1(\mathbf{x}) = 0.15(1+\sin(10x_1)) - x_2 \leq 0, \\
& g_2(\mathbf{x}) = (x_1-0.5)^2 - x_2 + 0.5 \geq 0, \\
& -1 \leq x_1 \leq 0, \\
& -1 \leq x_2 \leq 0
\end{aligned}
\tag{7}
$$

The problem 7 (Hickernell & Yuan, 1997) poses a difficult optimization challenge due to its very unequal distribution of optima, which includes 15 local minima and 8 internal maxima. Here, we examine the results of applying two nonlinear constraints and four limits on this function, beginning at $[-0.5, 0.5]$. Table 4 displays the outcomes to illustrate the efficacy of the algorithm. As can be seen in figure 7, the updated approach converges faster (in 8 iterations) and uses less processing time. This study aids algorithm selection for comparable limited optimization settings by providing significant insights into the efficacy of the algorithms for optimizing the limited Guilin Hills function.

Table 4. Performance Comparison & optimal outputs on Guilin hills Function

Method	$\mathbf{x}_{optimal}$	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	[0.4136 0.3834]	-1.067449	9.0421e-12	12	0.082022
KZoutendijk	[0.4135 0.3836]	-1.067450	9.0428e-12	8	0.057213

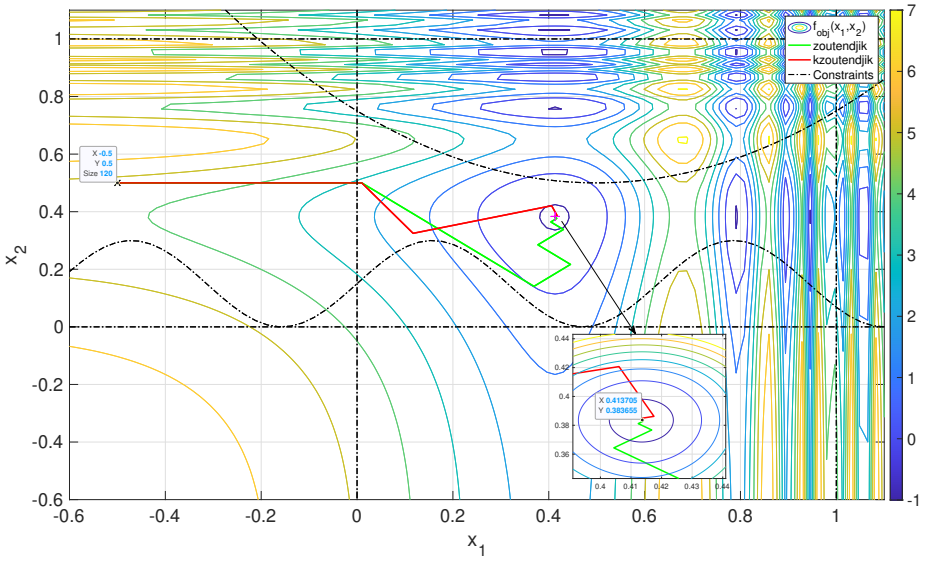


Figure 7. optimization of the Constrained Guilin hills Function: Zoutendijk vs. Kzoutendijk Algorithms

3.2. *Equality Constraints*

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2 \\ \text{subject to} \quad & g_1(\mathbf{x}) = x_1^2 + x_2^2 = 25, \\ & -5 \leq x_1 \leq 5, \\ & -5 \leq x_2 \leq 5 \end{aligned} \tag{8}$$

In the scenario depicted in Figure 8, the Himmelblau function is subject to one equality constraint and four bounds (Dasgupta et al., 2013). The algorithm's initial coordinates are $[-2, 0]$. Notable is the failure of the Zoutendijk algorithm to converge in this type of scenario. However, the modified algorithm converges within 33 iterations, demonstrating improved efficacy on equality constraints. The data table 5 summarises a thorough investigation into the effectiveness of both algorithms.

Table 5. Performance Comparison & optimal outputs on Himmelblau Function with equality constraints

Method	$\mathbf{x}_{optimal}$	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	[– –]	–	–	–	–
KZoutendijk	[-3.7793 -3.2832]	0.000000	0.000000	33	0.448013

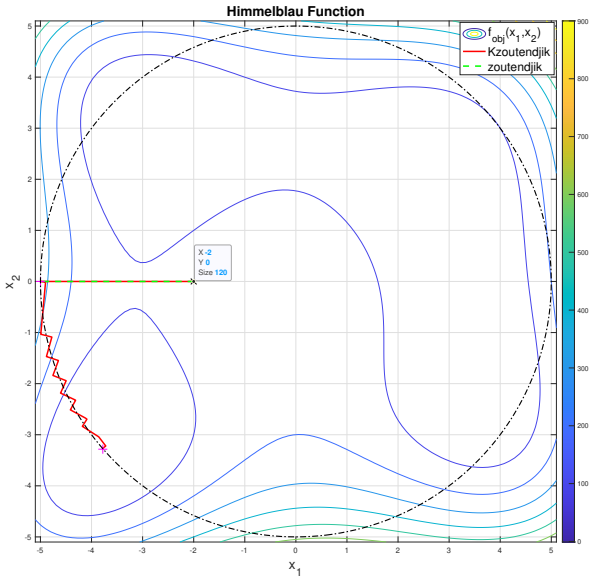


Figure 8. optimization of the Eq-Constrained Himmelblau Function: Zoutendijk vs. Kzoutendijk Algorithms

3.3. *Higher Dimensional Problems*

3.3.1. Seven Dimension Function

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = (x_1^2 - 10)^2 + 5(x_2 - 12)^2 + x_3^4 + 3(x_4 - 11)^2 + 10x_5^6 + 7x_6^2 + x_7^4 \\
& \quad - 4x_6x_7 - 10x_6 - 8x_7 \\
\text{subject to} \quad & g_1 = -127 + 2x_1^2 + 3x_2^4 + x_3 + 4x_4^2 + 5x_5 \leq 0, \\
& g_2 = -282 + 7x_1 + 3x_2 + 10x_3^2 + x_4 - x_5 \leq 0, \\
& g_3 = -196 + 23x_1 + x_2^2 + 6x_6^2 - 8x_7 \leq 0, \\
& g_4 = 4x_1^2 + x_2^2 - 3x_1x_2 + 2x_3^2 + 5x_6 - 11x_7 \leq 0, \\
& -10 \leq x_i \leq 10, \quad i = 1 \ldots 7
\end{aligned}
\tag{9}
$$

In the problem 9 (Mehta & Dasgupta, 2012), because of its seven-dimensional character and the participation of four nonlinear constraints, this problem poses a considerable and exciting challenge. The limited nature of the search space, limits the feasible region to an extremely small proportion of the total space (approximately 0.5%). This astounding discovery was initially published by Deb, 2000, who took into consideration the constraints that were placed on the variables.

In the performance evaluation conducted by Takahama and Sakai, 2005, simplex-based algorithms yielded a notable function value of 680.630 for addressing this issue. A detailed comparison of the performance of the Zoutendijk and KZoutendijk algorithms is presented in Table 6.

- Initial Point: $\mathbf{x}_0 = [1, 1, 1, 1, 1, 1, 1]$
- Initial point is feasible, but constraint boundaries emerge during iterations.
- Best known minima so far = 680.630
- Zoutendijk $\mathbf{x}_{optimal} = [2.8879, 2.1079, -0.3443, 3.6399, -0.3443, 0.1385, 1.8615]$
- KZoutendijk $\mathbf{x}_{optimal} = [2.6436, 1.9460, 0.0684, 4.2401, -0.6350, 0.7871, 1.8417]$

Table 6. Performance Comparison & optimal outputs on Seven dimensional Function

Method	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	649.395928	2.2952e-18	21	0.500486
KZoutendijk	639.660983	2.1084e-34	36	0.227631

3.3.2. Five Dimension Himmelblau's function

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = 5.3578547x_3^2 + 0.8356891x_1x_5 + 37.293239x_1 - 40792.141 \\
\text{subject to} \quad & g_{1,2} = 0 \leq 85.334407 + 0.0056858x_2x_3 + 0.000626x_1x_4 - 0.0022053x_3x_5 \leq 92, \\
& g_{3,4} = 90 \leq 80.51249 + 0.0071317x_2x_5 + 0.0029955x_1x_2 + 0.0021813x_3^2 \leq 110, \\
& g_{5,6} = 20 \leq 9.300961 + 0.00470276x_3x_5 + 0.0012547x_1x_3 + 0.0019085x_3x_4 \leq 25, \\
& 27 \leq x_k \leq 45, \quad k = 3, \ldots, 5 \\
& 78 \leq x_1 \leq 102, \\
& 33 \leq x_2 \leq 35
\end{aligned}
\tag{10}
$$

,

The problem 10 (Mehta & Dasgupta, 2012) consists of minimizing a function with five dimensions while adhering to six inequality constraints and five bounds, which was initially presented by Himmelblau, 1972. This problem has been investigated by some scholars which can be found listed in the appendix C.

In addition, several writers have evaluated their algorithms on a different version (Cagnina et al., 2008) of this problem, in which a particular value (0.0006262) in the constraint $g_{1,2}$ has been changed to 0.00026 (Mehta & Dasgupta, 2012). This variant of the problem was used to evaluate the authors' solutions. Researchers Fesanghary et al., 2008 & Coello, 2000 came up with optimum function values for this updated version of the equation, which were -31024.316 and -31,020.859, respectively.

Table 7 displays the optimal solutions achieved by both algorithms when utilizing randomly selected initial points.

- Initial Point: $\mathbf{x}_0 = [-79, 93, -100, 55, 64]$
- Initial point is infeasible.
- Best known minima so far = -31024.316
- Zoutendijk $\mathbf{x}_{optimal} = [78.0047, 34.9992, 27.0630, 44.9975, 44.9924]$
- KZoutendijk $\mathbf{x}_{optimal} = [78.0054, 34.9999, 27.0637, 44.9982, 44.9931]$

Table 7. Performance Comparison & optimal outputs on Five-dimensional Himmelblau's function

Method	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	-31026.027009	3.1084e-9	4	0.016760
KZoutendijk	-31025.725950	3.2952e-9	3	0.007314

3.3.3. Ten Dimension Function

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = x_1^2 + x_2^2 + x_1x_2 - 14x_1 - 16x_2 + (x_3 - 10)^2 + 4(x_4 - 5)^2 + (x_5 - 3)^2 \\
& + 2(x_6 - 1)^2 + 5x_7^2 + 7(x_8 - 11)^2 + 2(x_9 - 10)^2 + 2(x_{10} - 7)^2 + 45 \\
\text{subject to} \quad & g_1 = 4x_1 + 5x_2 - 3x_7 + 9x_8 \leq 105, \\
& g_2 = 10x_1 - 8x_2 - 17x_7 + 2x_8 \leq 0, \\
& g_3 = -8x_1 + 2x_2 + 5x_9 - 2x_{10} \leq 12, \\
& g_4 = 3(x_1 - 2)^2 + 4(x_2 - 3)^2 + 2x_3^2 - 7x_4 \leq 120, \\
& g_5 = 5x_1^2 + 8x_2 + (x_3 - 6)^2 - 2x_4 \leq 40, \\
& g_6 = x_1^2 + 2(x_2 - 2)^2 - 2x_1x_2 + 14x_5 - 6x_6 \leq 0, \\
& g_7 = 0.5(x_1 - 8)^2 + 2(x_2 - 4)^2 + 3x_5^2 - x_6 \leq 30, \\
& g_8 = -3x_1 + 6x_2 + 12(x_9 - 8)^2 - 7x_{10} \leq 0, \\
& -10 \leq x_k \leq 10, \quad k = 1, \ldots, 10
\end{aligned}
\tag{11}
$$

The problem 11 (Mehta & Dasgupta, 2012) has to be solved involves minimizing the value of a function with ten dimensions to its smallest possible value while adhering to 8 inequality constraints and 20 boundaries. KZoutendijk converges to a better optimum compared to the Zoutendijk algorithm. To provide an insightful comparison of the results obtained by the two algorithms, refer to Table 8.

- Initial Point: $\mathbf{x}_0 = [10, 1, 8, 7, 9, 4, 8, 9, 3, 6]$
- Initial point is infeasible.
- Best known minima so far = 24.3062091,
- Zoutendijk $\mathbf{x}_{optimal} = [2.6575, 2.0503, 7.6204, 7.1887, 2.0191, 4.1036, 7.9484, 9.0516, 6.4864, 6.2931]$
- KZoutendijk $\mathbf{x}_{optimal} = [2.2202, 2.6067, 7.8532, 6.1324, 1.7241, 3.3758, 3.2088, 9.9749, 7.4967, 6.4674]$

Table 8. Performance Comparison & optimal outputs on ten-dimensional function

Method	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	404.91	0.088045	7	0.802445
KZoutendijk	84.315307	2.0469e-11	32	11.661815

3.3.4. Thirteen Dimension Function

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) = 5(x_1 + x_2 + x_3 + x_4) - (x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} + x_{11} + x_{12} + x_{13}) \\
& - 5(x_1^2 + x_2^2 + x_3^2 + x_4^2) \\
\text{subject to} \quad & g_1 = 2x_1 + 2x_2 + x_{10} + x_{11} - 10 \leq 10, \\
& g_2 = 2x_1 + 2x_3 + x_{10} + x_{12} - 10 \leq 10, \\
& g_3 = 2x_2 + 2x_3 + x_{11} + x_{12} - 10 \leq 10, \\
& g_4 = -8x_1 + x_{10} \leq 0, \quad g_7 = -2x_4 - x_5 + x_{10}, \\
& g_5 = -8x_2 + x_{11} \leq 0, \quad g_8 = -2x_6 - x_7 + x_{11}, \\
& g_6 = -8x_3 + x_{12} \leq 0, \quad g_9 = -2x_8 - x_9 + x_{12}, \\
& 0 \leq x_k \leq 1, \quad k = 1, \ldots, 9 \ \& \ 13, \\
& 0 \leq x_k \leq 100, \quad k = 10, \ldots, 12
\end{aligned}
\tag{12}
$$

The problem 12 (Mehta & Dasgupta, 2012) at hand is finding the minimum value for a function in thirteen dimensions subject to six inequality constraints and twenty six limits. The outcomes of the two methods are compared in detail in Table 9.

- Initial Point: $\mathbf{x}_0 = [3, 1, 1, 2, 1, 2, 1, 1, 2, 4, 5, 6, 1]$
- Initial point is infeasible.
- Best known minima earlier Takahama and Sakai, 2005 = -15
- Zoutendijk $\mathbf{x}_{optimal} = [0.9999, 0.9999, 0.5376, 0.9999, 0.9999, 0.9999, 0.9999, 0.9999, 0.9999, 1.5699, 2.9945, 2.9998, 0.9999]$
- KZoutendijk $\mathbf{x}_{optimal} = [0.9999, 0.9999, 0.5346, 0.9999, 0.9999, 0.9999, 0.9999, 0.9999, 0.9999, 1.5698, 2.9945, 2.9997, 0.9999]$

Table 9. Performance Comparison & optimal outputs on thirteen-dimensional function

Method	$f(\mathbf{x}_{optimal})$	$\mathbf{E}_{KKT}$	Iteration	CPU time (s)
Zoutendijk	-12.509881	1.8655e-05	8	0.106147
KZoutendijk	-12.508087	2.383e-05	10	0.177239

4. Conclusion

In a nutshell, this paper presents convincing data that leads to the following key findings concerning the Kanpur version of the Zoutendijk method over the original Zoutendijk method and its influence on constrained optimization:

- In the majority of benchmark experiments, the proposed algorithm consistently outperformed the conventional Zoutendijk algorithm, demonstrating its superior performance and quicker convergence.
- Trust region strategy enables the algorithm to converge on feasible and refined solutions.
- The original Zoutendijk algorithm had trouble with equality constraints and usually didn't converge, but the proposed algorithm does better in this scenario.

In summary, the modified Zoutendijk algorithm is a robust and effective tool with great promise for optimization researchers and practitioners due to its superior performance and adaptability to various constraints.

References

Cagnina, L. C., Esquivel, S. C., & Coello, C. A. C. (2008). Solving engineering optimization problems with the simple constrained particle swarm optimizer. *Informatica*, *32*(3).

Coello, C. A. C. (2000). Use of a self-adaptive penalty approach for engineering optimization problems. *Computers in Industry*, *41*(2), 113–127.

Dasgupta, B., Divya, K., Mehta, V. K., & Deb, K. (2013). Repamo: Recursive perturbation approach for multimodal optimization. *Engineering Optimization*, *45*(9), 1073–1090.

Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer methods in applied mechanics and engineering*, *186*(2-4), 311–338.

Fesanghary, M., Mahdavi, M., Minary-Jolandan, M., & Alizadeh, Y. (2008). Hybridizing harmony search algorithm with sequential quadratic programming for engineering optimization problems. *Computer methods in applied mechanics and engineering*, *197*(33-40), 3080–3091.

Hickernell, F. J., & Yuan, Y.-x. (1997). A simple multistart algorithm for global optimization. *OR Transactions*, *1*(2), 1–11.

Himmelblau, D. (1972). Applied nonlinear optimization. *New York, US McGraw Hill*.

Homaifar, A., Qi, C. X., & Lai, S. H. (1994). Constrained optimization via genetic algorithms. *Simulation*, *62*(4), 242–253.

Lee, K. S., & Geem, Z. W. (2005). A new meta-heuristic algorithm for continuous engineering optimization: Harmony search theory and practice. *Computer methods in applied mechanics and engineering*, *194*(36-38), 3902–3933.

Mehta, V. K., & Dasgupta, B. (2012). A constrained optimization algorithm based on the simplex search method. *Engineering Optimization*, *44*(5), 537–550.

Pham, D. T., Haj Darwish, A., & Eldukhri, E. E. (2009). Optimisation of a fuzzy logic controller using the bees algorithm. *International Journal of Computer Aided Engineering and Technology*, *1*(2), 250–264.

Ragsdell, K., & Phillips, D. (1976). Optimal design of a class of welded structures using geometric programming.

Takahama, T., & Sakai, S. (2005). Constrained optimization by applying the/spl alpha/constrained method to the nonlinear simplex method with mutations. *IEEE Transactions on Evolutionary Computation*, *9*(5), 437–451.

Yıldız, A. R. (2008). Hybrid taguchi-harmony search algorithm for solving engineering optimization problems. *International Journal of Industrial Engineering*, *15*(3), 286–293.

Zoutendijk, G. (1966). Nonlinear programming: A numerical survey. *SIAM Journal on Control*, *4*(1), 194–210.
Zoutendijk, G. (1960). *Methods of feasible directions: A study in linear and non-linear programming*. Elsevier Publishing Company.

Appendix A. Zoutendijk Feasible Direction Method

$$\begin{aligned} &\underset{\forall x_i \in \mathbb{R}^n}{\text{minimize}} && f(\mathbf{x}) \\ &\text{subject to} && g(\mathbf{x}) \leq 0 \end{aligned} \tag{A1}$$

(1) In the k-th iteration, if the current point is interior, the algorithm moves along the steepest descent direction until constraints are encountered.

$$\mathbf{d} = -\nabla f(\mathbf{x}_k)$$

(2) At a given point $\mathbf{x}_k$, if the current point lies on the constraint surface, represented by the active set J of active constraints with a cardinality of m ($m \leq n$), the algorithm searches for a feasible direction to identify the optimal solution. It then solves an LP sub-problem to find the descending feasible direction.

$$\begin{aligned} \underset{\mathbf{d},\beta}{\text{minimize}} \quad & -\beta \\ \text{subject to} \quad & \nabla f^T \mathbf{d} \leq -\beta, \\ & \nabla g_j^T \mathbf{d} \leq -\beta \quad \forall j \in J \ \ \& \ \ j = 1, 2 \ldots m, \\ & -1 \leq d_i \leq 1 \quad \forall i = 1, 2 \ldots n, \\ & \beta \geq 0 \end{aligned} \tag{A2}$$

(3) We are dealing with $n + 1$ variables that are subject to $2n + m + 2$ constraints.
(4) Afterwards, a line search is conducted along $\mathbf{d}$ using methods such as the golden section or interval halving method to determine an appropriate value for α,

$$\alpha = \min_{\alpha} f(\mathbf{x} + \alpha \mathbf{d}) \quad \text{where, } \alpha \in (0, \alpha^u)$$

(5) This value is then utilized to compute the next point using the formula,

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \mathbf{d}$$

Continue the process until a KKT point is found, *i.e* when the value of the $\|E_{KKT}\|$ becomes zero.

Appendix B. KKT error vector

Using the KKT criteria, which include constraint values, and entries of ∇L and $\boldsymbol{\mu}$, we create an error vector that is used to evaluate and report the precision of a result point.

The magnitude of this vector represents the degree of inaccuracy around a local minimum (technically a KKT point).

$$\mathbf{E}_{KKT} = \begin{bmatrix} h \\ \max(0, \mathbf{g}) \\ \max(0, -\mu) \\ \mu_j g_j \\ \nabla L \end{bmatrix} \quad \in \mathbb{R}^{n+3m} \;\&\; j = 1, 2 \ldots m \tag{B1}$$

Where, $\mathbf{x} \in \mathbb{R}^n, \mathbf{g} \in \mathbb{R}^m \;\&\; \|E_{KKT}\| = 0$ represents a KKT point.

Appendix C. Algorithms survey

Table C1. A survey of different algorithms.

Authors	Algorithm/Method
Takahama and Sakai, 2005	α Constrained Method with Mutation
Deb, 2000	Genetic Algorithm
Ray and Liew (2003)	Algorithm based on Simulation of Social Behaviour
Coello, 2000	GA + Self Adaptive Penalty Approach
Mezura and Coello, 2000	Evolutionary Algorithm
Coello, 2000	Evolutionary Multi-Objective optimization
Ragsdell and Phillips, 1976	Geometric Programming
Lee and Geem, 2005	Meta-heuristic based on Harmony Search
Fesanghary et al., 2008	Hybrid Method (Harmonic search + SQP)
Homaifar et al., 1994	Genetic Algorithm
Yıldız, 2008	Harmony Search
Pham et al., 2009	Bees Algorithm
Cagnina et al., 2008	Particle Swam optimizer